

1. Process Concept, Concurrency & Race Conditions

Syllabus Coverage: Gateway Classes Slides 3–15 | Process Concepts, Concurrency Principles, Independent vs Cooperating Processes, Race Conditions.

1.1 Process Concept & Memory Organization

Program in Execution: Hard disk par program ek passive byte file hota hai. Jab OS isse RAM mein load karta hai, toh yeh **active process** ban jata hai jiske paas apna context aur resource allocation hota hai.

- **Text Section:** Program instructions (code segment).
- **Data Section:** Global aur static variables.
- **Heap:** Dynamic runtime memory (via `malloc`).
- **Stack:** Local activation records, function parameters aur return addresses.
- **Process Control Block (PCB):** Kernel data structure containing PID, Program Counter, CPU registers, scheduling priority, aur open file descriptors.

1.2 Independent vs Cooperating Processes

Parameter	Independent Process	Cooperating Process
Shared State	Koi shared data ya resource nahi hota.	Shared variables, buffer, files ya hardware resources.
Execution Effect	Kisi doosre process ko affect nahi karta.	Doosre process ke execution ko directly modify kar sakta hai.
Determinism	Deterministic: Same input = Always same result.	Non-deterministic: Outcome depends on execution order.
Risk	Zero synchronization hazard.	Race Condition & Data Inconsistency risk!

1.3 Race Condition Execution Trace

Jab concurrent processes bina synchronization ke shared data par read/write karte hain, toh outcome interleaved execution order par depend karta hai.

```
; Initial Shared Variable: Balance = 1000
; Process P1 (Deposit 500)
1. LOAD  R1, Balance    ; R1 = 1000
2. ADD   R1, 500        ; R1 = 1500
; --- Context Switch to P2 ---
3. LOAD  R2, Balance    ; R2 = 1000 (stale value!)
4. SUB   R2, 200        ; R2 = 800
5. STORE Balance, R2    ; RAM Balance = 800
; --- Context Switch back to P1 ---
6. STORE Balance, R1    ; RAM Balance = 1500! (Withdrawal lost!)
```

Process Concurrency & Race Condition Architecture

Independent vs Cooperating Processes & Data Inconsistency

1. Independent Processes (Deterministic)

- Does NOT share memory, variables or I/O state.
 - Execution sequence does not affect other processes.
 - Deterministic output: Same input = Always same output.
- Example:** Calculator and Music Player running simultaneously.

✓ No Synchronization Required

2. Cooperating Processes (Non-Deterministic)

- Shares state, shared memory, files, or variables.
 - Can affect or be affected by other concurrent processes.
 - Non-deterministic: Order of execution dictates final output.
- Example:** Producer-Consumer, ATM balance concurrent withdraw.

⚠ Prone to Race Condition & Data Inconsistency!

⚠ The Race Condition: Bank Account Balance = 1000

Process P1 (Deposit 500)

R1 = Balance (1000)
R1 = R1 + 500 (1500)
[Preempted before store!]
Later: Balance = 1500

Shared Memory: Balance
Initial Value: 1000

Final Corrupt Value: 800 or 1500!

Process P2 (Withdraw 200)

R2 = Balance (1000)
R2 = R2 - 200 (800)
Balance = 800 (Stored!)
P1 overwrites later!

1.4 Exam & Interview Summary

- **Core Rule:** Shared data + Preemption = Race Condition.
- **Solution Required:** Mutual Exclusion around shared code (Critical Section).